Programowanie współbieżne w C++

Lista zadań nr 8

Na pracownię 23 kwietnia 2026

Zadanie 1 (7+7pkt) Zastosowanie symulatora CppMem.

Spośród swoich rozwiązań zadań listy 7 wybierz dowolne dwa, uprość je pozbywając się detali niezwiązanych z dostępami do pamięci i zasymuluj w CppMem. Upewnij się, że nie występują wyścigi danych, a dostępy do współdzielonych lokacji są odpowiednio zsynchronizowane. Przekonaj się dzięki temu, że dostępy do zmiennych współdzielonych są w Twoich programach wykonywane są poprawnie. Wyniki swojej pracy spisz w postaci raportu z załączonymi zrzutami ekranu z symulatora.

Wskazówka: Pętle nieskończone zamień na pętle o skończonej, małej liczbie iteracji (równej np 1 lub 2), użyj funkcji readsvalue do wymuszenia synchronizacji tam, gdzie pętla ma oczekiwać na konkretną wartość. Współdzielone zmienne uczyń globalnymi. Zamiast wpisywać do pamięci dowolne wartości, wpisuj stałe. Tam, gdzie wątków może być dowolnie wiele, zasymuluj 2 lub 3.

Zadanie 2 (6pkt) Klasa TransactionConsensus. Zastosowanie sekwencji zwalniań (release sequence).

Mamy system współbieżny, w którym jeden wyróżniony wątek tworzy (pojedyńczą) transakcję. Następnie wiele wątków może na tą transakcję poprzeć lub nie. Ostatecznie, inny wyróżniony wątek sprawdza, czy transakcja otrzymała wymaganą liczbę głosów poparcia i jeśli tak, to wykonuje ją. Uzupełnij poniższy kod, implementując funkcje realizowane przez te trzy rodzaje wątków. Zaimplementuj również kod testujący, który realizuje powyżej opisany protokół tworzenia, głosowania i wykonywania transakcji.

```
#include <atomic>
#include <string>
#include <iostream>
#include <thread>
#include <vector>

class TransactionConsensus {
private:
    std::string payload;     // Zwykłe dane, brak atomowości
    double amount;           // Zwykłe dane, brak atomowości

    // Stan: 0 oznacza brak propozycji, >0 oznacza liczbę głosów poparcia
```

```
    std::atomic<uint32_t> votes{0};

public:
    // Wątek inicjujący publikuje propozycję transakcji
    void propose(const std::string& p, double a) {
        payload = p;
        amount = a;
        // 1. ZADANIE: Ustaw 'votes' na 1. Użyj najsłabszego,
        // odpowiedniego porządku pamięci, który opublikuje 'payload' i
'amount'.
        // MIEJSCE NA TWÓJ KOD
    }

    // Wątki robocze dodają swoje poparcie (wywoływane współbieżnie)
    void vote_up() {
        // 2. ZADANIE: Zwiększ 'votes' o 1.
        // Zastosuj odpowiednią operację atomową, np. RMW
(Read-Modify-Write).
        // MIEJSCE NA TWÓJ KOD
    }

    // Wątek egzekutora próbuje wykonać transakcję
    bool try_execute(std::string& out_p, double& out_a, uint32_t&
out_level) const {
        // 3. ZADANIE: Odczytaj 'votes'. Użyj odpowiedniego porządku
        // synchronizującego z publikacją.
        // MIEJSCE NA TWÓJ KOD
        uint32_t level = /* ... */;

        if (level > 0) {
            out_p = payload;
            out_a = amount;
            out_level = level;
            return true;
        }
        return false;
    }
};
```

Zaimplementuj dwa warianty działania metody vote_up: 1. metoda “ślepo” popiera transakcję, tj. nie odczytuje zmiennych payload ani amount i 2. metoda odczytuje te zmienne i na ich podstawie decyduje o oddaniu głosu. W obydwu przypadkach użyj najsłabszych możliwych porządków dostępu do atomowych zmiennych współdzielonych tak, by Twój program nie generował wyścigów danych. Dla obydwu wariantów 1. i 2. zastanów się czy możesz w metodzie vote_up użyć: a) atomowej inkrementacji zmiennej votes, b) pętli z instrukcją CAS, c) atomowych instrukcji load/store w następujący sposób `auto tmp = consensus_level.load(std::memory_order_relaxed); consensus_level.store(tmp + 1, std::memory_order_relaxed);`.